초급반 3주차

이O재 @ AlKor, CYDF

목차

- 복습
- 반복문
- 배열
- 문자열
- 해킹 (buffer overflow attack)
- 연습문제

복습 - 2753

```
#include <bits/stdc++.h>

int main() {
    int year;
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0)
        || year % 400 == 0) {
        printf("1\n");
    } else {
        printf("0\n");
    }
}
```

복습 - 14681

```c
#include <bits/stdc++.h>

int main() {
    int x, y;
    scanf("%d %d", &x, &y);

    if (x > 0) {
        if (y > 0) {
            printf("1\n");
        } else {
            printf("4\n");
        }
    }
```

```c
    else {
        if (y > 0) {
            printf("2\n");
        } else {
            printf("3\n");
        }
    }
}
```

복습 - 2884

```
#include <bits/stdc++.h>

int main() {
    int h, m;
    scanf("%d %d", &h, &m);

    if (m >= 45) {
        m -= 45;
    } else {
        m += 15;
        h = (h - 1 + 24) % 24;
    }
    printf("%d %d\n", h, m);
}
```

복습 - 2480

```
#include <bits/stdc++.h>

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    if (a == b && b == c) {
        printf("%d\n", 10000 + a * 1000);
    } else if (a == b || b == c || a == c) {
        printf("%d\n", 1000 + (a == b ? a : (b == c ? b : c)) * 100);
    } else {
        printf("%d\n", (a > b ? (a > c ? a : c) : (b > c ? b : c)) * 100);
    }
}
```

복습 - 2480

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;

    if (a == b && b == c) {
        printf("%d\n", 10000 + a * 1000);
    } else if (a == b || b == c || a == c) {
        printf("%d\n", 1000 + (a == b ? a : (b == c ? b : c)) * 100);
    } else {
        printf("%d\n", max({a, b, c}) * 100);
    }
}
```

복습 - 27294

```
#include <bits/stdc++.h>

int main() {
    int t, s;
    scanf("%d %d", &t, &s);

    if (12 <= t && t <= 16 && s == 0) {
        printf("320\n");
    } else {
        printf("280\n");
    }
}
```

복습 - 2525

```
#include <bits/stdc++.h>

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    b += c;
    a = (a + b / 60) % 24;

    printf("%d %d\n", a, b % 60);
}
```

반복문

- 반복문이란?
  - 어떤 작업을 반복하기 위해 코드를 여러 번 실행하도록 하는 구조

- 반복문 예시

```
printf("Hello, World\n");
printf("Hello, World\n");
printf("Hello, World\n");
printf("Hello, World\n");
printf("Hello, World\n");
```

```
for (int i = 0; i < 5; i++) {
    printf("Hello, World\n");
}
```

반복문

- for문
  - 가장 많이 쓰이는 반복문
  - 일반적으로 반복 횟수를 설정할 때 사용함

- while문
  - 조건이 참인 동안 반복할 때 쓰이는 반복문

- do while문
  - 구조적으로 while문과 동일하나, 적어도 한 번 실행이 보장된 반복문

반복문

- for문의 구조

```
for (<initial>; <condition>; <change>) {
    // code
}
```

- initial : 반복에 사용할 변수의 값을 초기화해줌
- condition: 반복을 언제까지 실행할지 조건을 설정함
- change : 변수의 값을 변화시킴

반복문

- 반복문 동작 과정

```
for (int i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}
```

- 내부 과정: i = 0 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 1

- 출력 값

```
0. Hello World
```

반복문

- 반복문 동작 과정

```
for (int i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}
```

- 내부 과정: i = 1 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 2

- 출력 값

```
0. Hello World
1. Hello World
```

반복문

- 반복문 동작 과정

```
for (int i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}
```

- 내부 과정: i = 2 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 3

- 출력 값

```
0. Hello World
1. Hello World
2. Hello World
```

반복문

- 반복문 동작 과정

```
for (int i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}
```

- 내부 과정: i = 3 ➔ i < 3 (false) ➔ stop ➔ for문 종료

- 출력 값

```
0. Hello World
1. Hello World
2. Hello World
```

반복문

- for문에서의 변수

```
for (int i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}

printf("%d. Hello World\n", i); // error
```

- for문 내부에서 선언한 변수는 for문 바깥에서 사용 불가
  - 나중에 전역변수와 지역변수 개념을 배울 때 자세한 설명

```
int i;

for (i = 0; i < 3; i++) {
    printf("%d. Hello World\n", i);
}

printf("%d. Hello World\n", i);
```

반복문

- for문의 제어

```
for (int i = 0; i < 20; i++) {
    if (i % 2 == 0) continue;
    if (i > 15) break;

    printf("%d ", i);
}
```

- continue : for문의 시작으로 돌아감
- break : for문을 강제 종료함

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 1; i < 20; i += 2) {
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 1; i < 20; i += 2) {
    printf("%d ", i);
}
```

```
1 3 5 7 9 11 13 15 17 19
```

```
for (int i = 0; i < 20; i++) {
    if (i % 2 == 1) printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 20; i < -1; i -= 3) {
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 20; i < -1; i -= 3) {
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 20; i > -1; i -= 3) {
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
for (int i = 20; i > -1; i -= 3) {
    printf("%d ", i);
}
```

```
20 17 14 11 8 5 2
```

```
for (int i = 20; i > -1; i--) {
    if (i % 3 == 2) printf("%d ", i);
}
```

반복문

- for문의 다양한 활용

```
int i = 0;

for (; i <= 10; i+=2) {
    printf("%d ", i);
}
```

```
for (int i = 0; i <= 10; ) {
    printf("%d ", i);
    i += 2;
}
```

```
for (int i = 0; ; ) {
    printf("%d ", i);
    i += 2;
}
```

반복문

- for문의 다양한 활용

```
for (int i = 0; i <= 10; i++) printf("%d ", i);
```

```
for (int i = 0; i <= 10; i++)
    printf("%d ", i);
    printf("Hello", i);
```

```
for (int i = 0, j = 0; i <= 10; i++, j += 2) {
    printf("%d %d\n", i, j);
}
```

반복문

- 정수 하나를 입력 받아, 1부터 입력 받은 수 까지 3의 배수의 합을 구하자

반복문

- 정수 하나를 입력 받아, 1부터 입력 받은 수 까지 3의 배수의 합을 구하자

```
#include <bits/stdc++.h>

int main() {
    int a, sum = 0;
    scanf("%d", &a);

    for (int i = 1; i <= a; i++) {
        if (i % 3 == 0) sum += i;
    }

    printf("%d\n", sum);
}
```

반복문

- while문의 구조

```
while (<condition>) {
    // code
}
```

- condition: 반복을 언제까지 실행할지 조건을 설정함

반복문

- 반복문 동작 과정

```
int i = 0;

while (i < 3) {
    printf("%d. Hello World\n", i);
    i++;
}
```

- 내부 과정: i = 0 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 1

- 출력 값

```
0. Hello World
```

반복문

- 반복문 동작 과정

```
int i = 0;

while (i < 3) {
    printf("%d. Hello World\n", i);
    i++;
}
```

- 내부 과정: i = 1 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 2

- 출력 값

```
0. Hello World
1. Hello World
```

반복문

- 반복문 동작 과정

```
int i = 0;

while (i < 3) {
    printf("%d. Hello World\n", i);
    i++;
}
```

- 내부 과정: i = 2 ➔ i < 3 (true) ➔ printf ➔ i++ ➔ i = 3

- 출력 값

```
0. Hello World
1. Hello World
2. Hello World
```

반복문

- 반복문 동작 과정

```
int i = 0;

while (i < 3) {
    printf("%d. Hello World\n", i);
    i++;
}
```

- 내부 과정: i = 3 ➔ i < 3 (false) ➔ stop ➔ while문 종료

- 출력 값

```
0. Hello World
1. Hello World
2. Hello World
```

반복문

- 다음 코드의 출력 값 예상하기

```
int i = 5;

while (i > 0) {
    printf("%d ", i);
    i--;
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
int i = 5;

while (i > 0) {
    printf("%d ", i);
    i--;
}
```

```
5 4 3 2 1
```

```
int i = 5;

while (i >= 0) {
    i--;
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
int i = 0;

while (i < 10) {
    printf("%d ", i);
}
```

반복문

- 다음 코드의 출력 값 예상하기

```
int i = 0;

while (i < 10) {
    printf("%d ", i);
}
```

```
0 0 0 0 0 0 0...
```

반복문

- while문의 다양한 활용

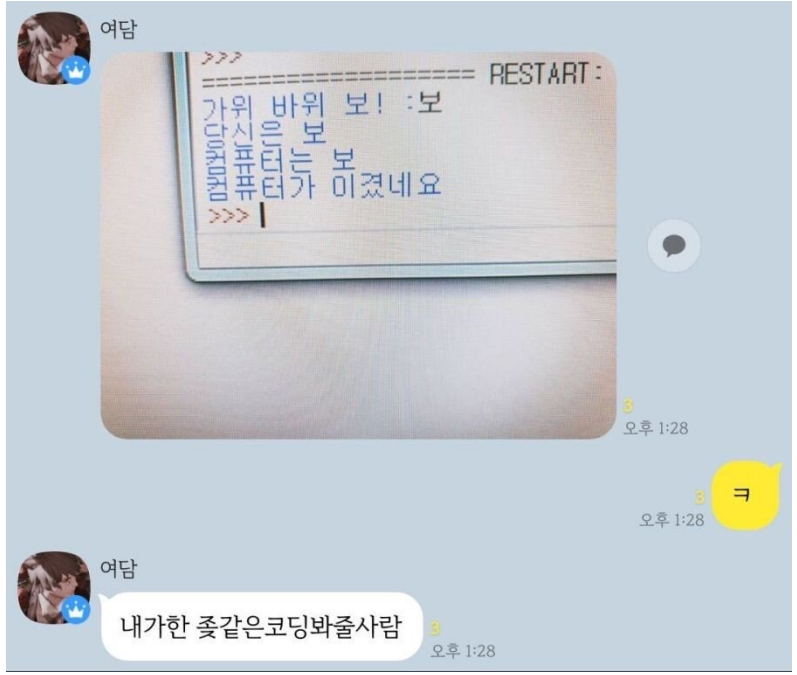
```
while (true) {
    printf("Hello\n");
}
```

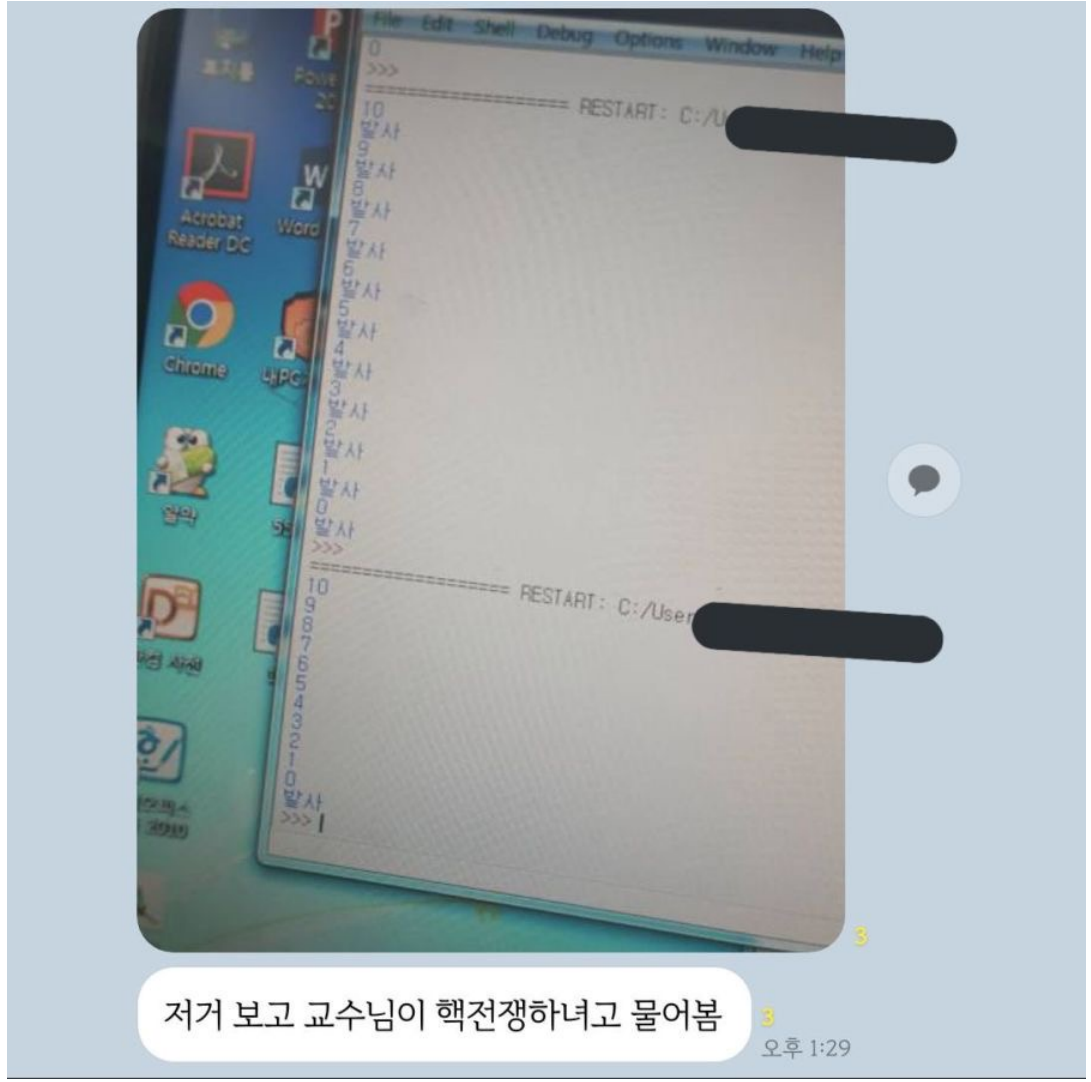
```
while (true) {
    int a;
    scanf("%d", &a);

    if (a % 2 == 0) break;

    printf("짝수를 입력할 때 까지 반복합니다.\n");
}
```

반복문

- 10 ~ 0까지 카운트다운 후 로켓 발사하는 프로그램 제작하기

- 제약 조건: while문 사용





발사발사발사발사 히히

반복문

```
#include <bits/stdc++.h>

int main() {
    int count = 11;

    while (count--) {
        printf("%d\n", count);
    }
    printf("발사");
}
```

반복문

- do while문의 구조

```
do {
    // code
} while(<condition>);
```

- condition: 반복을 언제까지 실행할지 조건을 설정함

- do while문을 언제 사용할까?
  - id/pw를 입력할 때

배열

- 배열이란?
  - 같은 자료형의 변수를 일렬로 늘어놓은 구조

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

- 배열을 사용하는 경우
  - 학생들의 점수를 저장할 때
  - 데이터를 반복하면서 접근할 때

- 배열은 주로 반복문과 함께 사용

배열

- 배열의 선언

```
<type> <name>[<size>] (= <initial value>);
int a[10];
```

- 배열의 각 요소 접근

```
<name>[<index>]
a[9]
```

- 인덱스

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70	80	90	100

배열

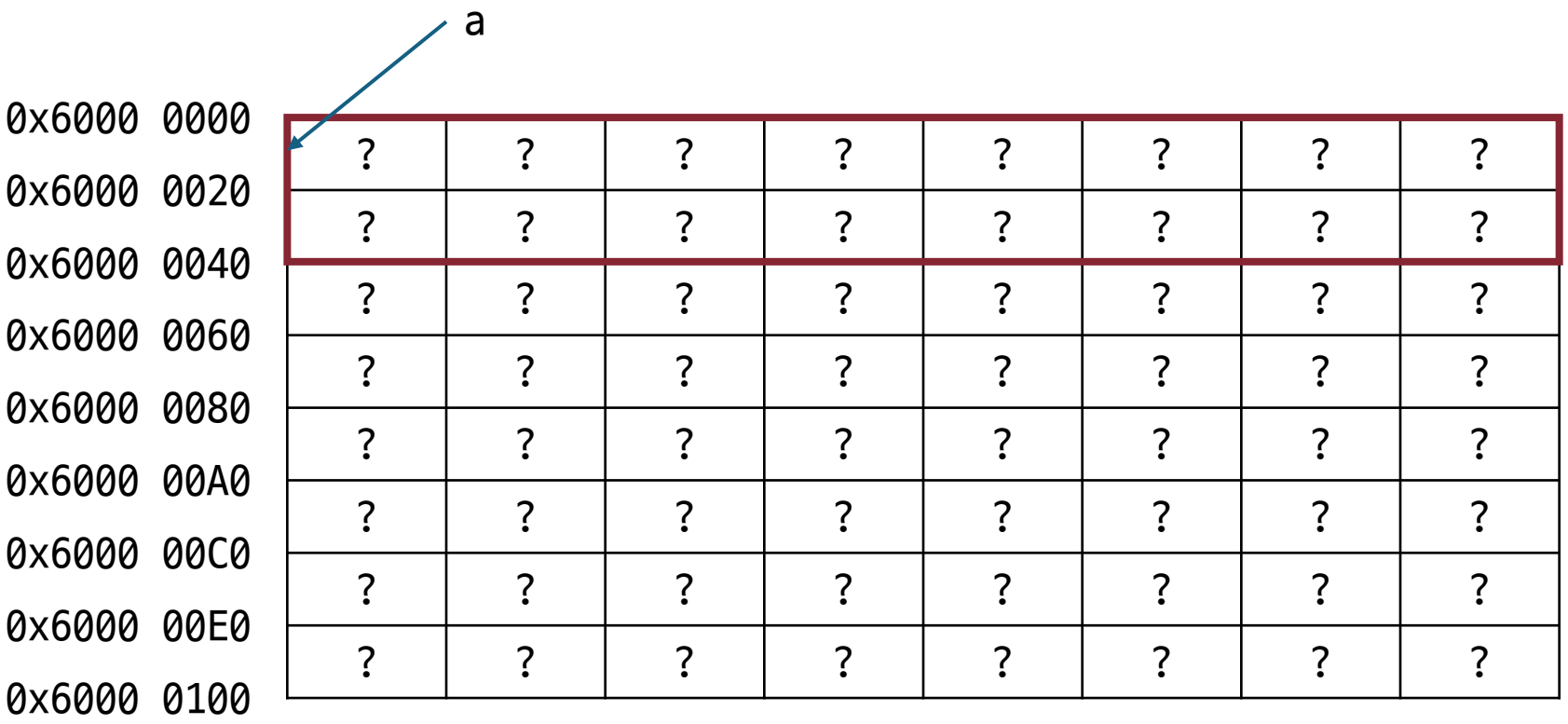
- 배열의 저장 방법

0x6000 0000	?	?	?	?	?	?	?	?
0x6000 0020	?	?	?	?	?	?	?	?
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100								

배열

- 배열의 저장 방법

```
int a[16];
```



배열

- 배열의 저장 방법

```
int a[16] = {1, 2, 3};
```

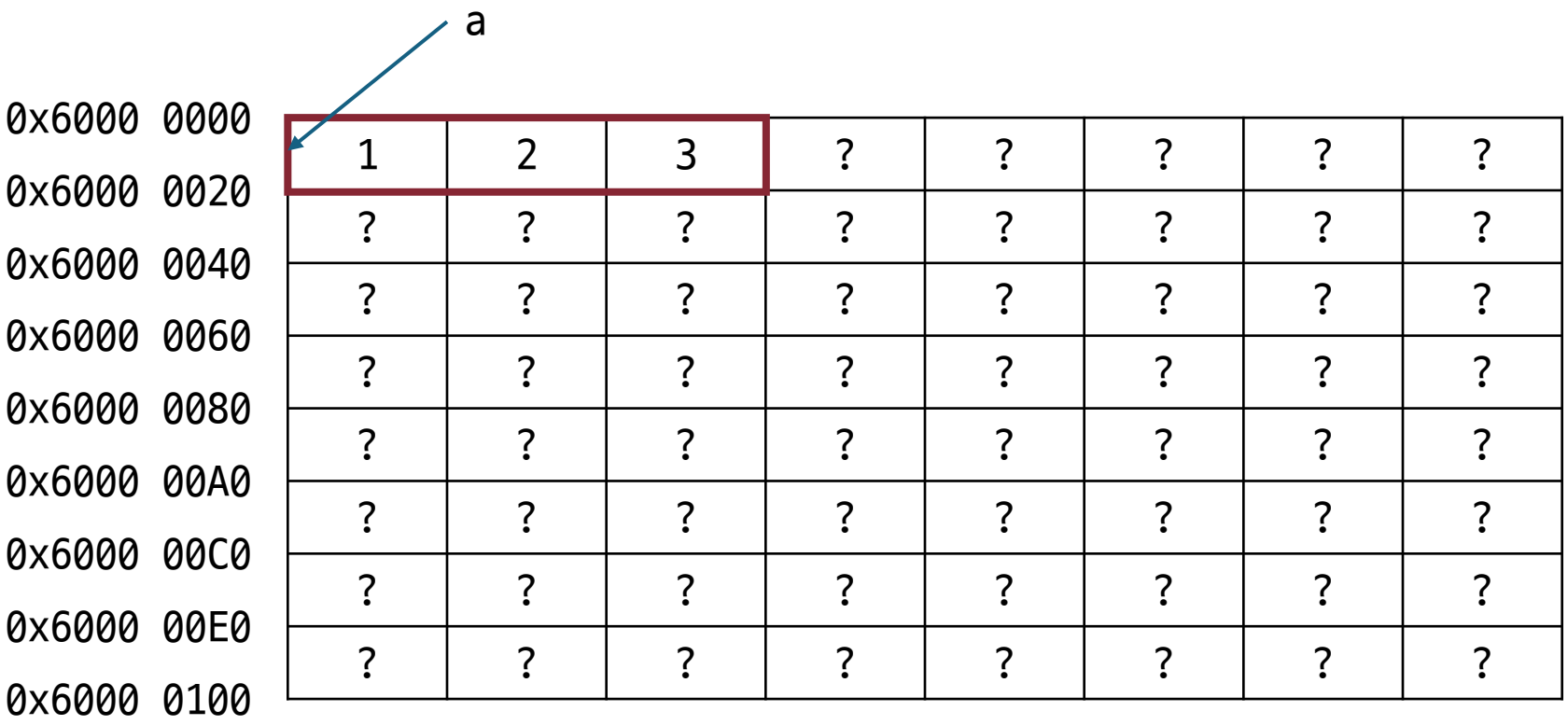
a

0x6000 0000	1	2	3	0	0	0	0	0
0x6000 0020	0	0	0	0	0	0	0	0
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100								

배열

- 배열의 저장 방법

```
int a[] = {1, 2, 3};
```



배열

- 배열의 저장 방법

```
int a[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
```

a

0x6000 0000	1	2	3	4	5	6	7	8
0x6000 0020	9	10	11	12	13	14	15	16
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100	?	?	?	?	?	?	?	?

배열

- 배열의 원소 접근 방법

```
int a[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
a[15] = 17; // (0x6000 0000 + 0xF)
```

a

0x6000 0000	1	2	3	4	5	6	7	8
0x6000 0020	9	10	11	12	13	14	15	17
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100	?	?	?	?	?	?	?	?

배열

- 배열의 원소 접근 방법

```
int a[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
a[16]; // ?
a[16] = 17; // ?
```

a

0x6000 0000	1	2	3	4	5	6	7	8
0x6000 0020	9	10	11	12	13	14	15	16
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100	?	?	?	?	?	?	?	?

배열

- 배열의 원소 접근 방법

```
int a[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
int b;
scanf("%d", &b); // scanf("%d", &a[16]);
```

a

&b

0x6000 0000	1	2	3	4	5	6	7	8
0x6000 0020	9	10	11	12	13	14	15	16
0x6000 0040	?	?	?	?	?	?	?	?
0x6000 0060	?	?	?	?	?	?	?	?
0x6000 0080	?	?	?	?	?	?	?	?
0x6000 00A0	?	?	?	?	?	?	?	?
0x6000 00C0	?	?	?	?	?	?	?	?
0x6000 00E0	?	?	?	?	?	?	?	?
0x6000 0100	?	?	?	?	?	?	?	?

배열

- 배열에 입력 받는 방법

- 예시
  - 첫 번째 줄에 총 입력할 수의 개수 N 입력 (N <= 100)
  - 두 번째 줄에 N개의 수를 공백으로 구분하여 입력

  - N개의 수를 차례로 출력

```
int n, a[100];
scanf("%d", &n);
for (int i = 0; i < 100; i++) scanf("%d", &a[i]);

for (int i = 0; i < 100; i++) printf("%d ", a[i]);
```

배열

- C++에서는 STL(Standard Template Library)에 vector 존재

- vector의 선언

```
vector<type> <name>(<size>);
vector<int> a(10);

vector<type> <name>(<size>, <initial value>);
vector<int> a(10, -1);

vector<type> <name> = {<values>};
vector<int> a = {1, 2, 3};
```

배열

- vector의 장점
  - 요소를 유동적으로 추가 가능

```
vector<int> a(5, 0);
a.push_back(1); // a = {0, 0, 0, 0, 0, 1}
```

  - 요소를 쉽게 탐색 가능

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    vector<int> a(5);
    iota(a.begin(), a.end(), 1); // a = {1, 2, 3, 4, 5};
    for (auto i: a) cout << i << ' ';
    cout << '\n';
}
```

문자열

- char로 이루어진 배열

```
char a[10] = “Hello”;
```

0	1	2	3	4	5	6	7	8	9
H	e	l	l	o	\0				

- 문자열의 마지막은 항상 \0(Null)이 자동으로 들어감
  - 항상 Null이 들어갈 여유 공간이 필요함. 즉 아래와 같이 선언하면 안됨

```
char a[8] = "Hi";
char b[8];

scanf("%s", b); // Hello!!! 입력
printf("%s %s\n", a, b);
```

문자열

- scanf(“%s”, a);의 취약점
  - 위 코드는 입력 받는 길이를 제한할 수 없음(이전 페이지 참고)

```
char a[8] = "Hi";
char b[8];

scanf("%s", b); // Hello!!! 입력
printf("%s %s\n", a, b);
```

	0	1	2	3	4	5	6	7
b →	H	e	l	l	o	\0		
a →	H	i	\0					

	0	1	2	3	4	5	6	7
b →	H	e	l	l	o	!	!	!
a →	\0	i	\0					

문자열

- scanf(“%s”, a);의 취약점
  - 위 코드는 입력 받는 길이를 제한할 수 없음(이전 페이지 참고)

```
char a[8] = "Hi";
char b[8];

scanf("%s", b); // Hello!!!World 입력
printf("%s %s\n", a, b); // a = “World”, b = “Hello!!!World”
```

	0	1	2	3	4	5	6	7
b →	H	e	l	l	o	!	!	!
a →	W	o	r	l	d	\0		

- a의 값이 바뀌는 것을 확인할 수 있음 ➔ buffer overflow attack (해킹)

buffer overflow attack

- 아래 코드를 생각해보자.

```
#include <bits/stdc++.h>

int main() {
    int is_admin = 0;
    char id[16];

    scanf("%s", id);

    if (is_admin) {
        printf("admin 권한 획득");
    } else {
        printf("일반 유저 입니다.");
    }
}
```

buffer overflow attack

- 적당히 긴 문자열 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!을 넣으면?
  - admin 권한 획득할 수 있는 것을 확인

- 이 공격을 막기 위한 방법
  - scanf("%s");를 사용하지 않는다. ➔ 안전한 함수 사용 (Visual Studio scanf_s)
  - 메모리 배치를 랜덤하게 한다.
  - C++ string을 사용한다.

문자열

- C++ string의 선언

```
string <name>;
string s;
```

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    string s, word;

    getline(cin, s);
    stringstream ss(s);

    while (ss >> word) {
        cout << word << '\n';
    }
}
```

문자열

- C 언어에서 띄어쓰기 포함해서 입력 받는 방법

```
char a[100];
scanf(“%[^\n]s”, a);
```

문자열

- 입력 받은 문자의 길이를 출력하는 코드

문자열

```c
#include <bits/stdc++.h>

int main() {
    char a[100];
    scanf("%s", a);

    int len = 0;
    for (; a[len] != '\0'; len++);
    printf("%d\n", len);
}

int main() {
    char a[100];
    scanf("%s", a);
    printf("%d\n", strlen(a));
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    string a;
    cin >> a;
    cout << a.size() << '\n';
}
```

문자열

- 입력 받은 두 문자가 같으면 “Yes”, 다르면 “No” 출력하는 코드

문자열

```c
#include <bits/stdc++.h>

int main() {
    char a[100], b[100];
    scanf("%s %s", a, b);

    int flag = 0, i = 0;

    for (; a[i]; i++) {
        if (a[i] != b[i]) {
            flag = 1;
        }
    }
```

```c
    if (b[i] == '\0' && flag == 0) {
        printf("Yes\n");
    } else {
        printf("No\n");
    }
}
```

문자열

```c
#include <bits/stdc++.h>

int main() {
    char a[100], b[100];
    scanf("%s %s", a, b);

    if (strcmp(a, b) == 0) {
        printf("Yes\n");
    } else {
        printf("No\n");
    }
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    string a, b;
    cin >> a >> b;

    if (a == b) {
        cout << "Yes\n";
    } else {
        cout << "No\n";
    }
}
```

연습문제

- 숙제

- 난이도 하: 10807, 10871, 10818, 2562, 11654, 10809, 5622

- 난이도 중: 1157, 2941, 2581, 1193

- 난이도 상: 11659

- 난이도 ??: 33486